Homework 02

Steven Jia

September 6, 2026

Question 1

(a) Simulation

I made the design matrix once at the start and did not make it again inside the loop. Each run got a new training response and a new test response. I used lm.fit for all the models.

```
set.seed(43202)

n <- 100
p <- 20
B <- 200

X_all <- matrix(rnorm(n * p), nrow = n, ncol = p)
beta <- 0.4^(sqrt(1:p))
mu <- as.vector(X_all %*% beta)

make_X <- function(m) {
  if (m == 0) {
    matrix(1, nrow = n, ncol = 1)
  } else {
    cbind(1, X_all[, 1:m, drop = FALSE])
  }
}

train_mse <- matrix(NA_real_, B, p + 1)
test_mse <- matrix(NA_real_, B, p + 1)
one_test_mse <- numeric(p + 1)

for (b in 1:B) {
  y <- mu + rnorm(n)
  y_test <- mu + rnorm(n)

  for (m in 0:p) {
    fit <- lm.fit(make_X(m), y)
    y_hat <- fit$fitted.values
    train_mse[b, m + 1] <- mean((y - y_hat)^2)
    test_mse[b, m + 1] <- mean((y_test - y_hat)^2)
  }

  if (b == 1) {
    one_test_mse <- test_mse[b, ]
  }
}

violations <- sum(apply(
  train_mse, 1,
```

```
  function(z) any(diff(z) > 1e-10)
))

avg_train <- colMeans(train_mse)
avg_test <- colMeans(test_mse)

cat("Runs where training MSE increased:", violations, "\n")
```

```
## Runs where training MSE increased: 0
```

```
cat("Smallest average test MSE is at m =",
    which.min(avg_test) - 1, "\n")
```

```
## Smallest average test MSE is at m = 6
```

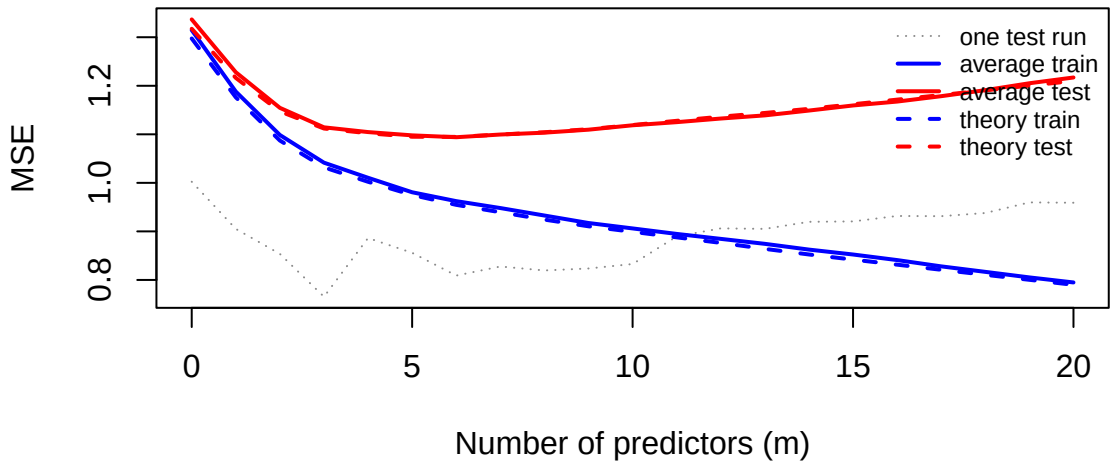
The check gave 0. So none of the 200 runs had a training MSE increase. This makes sense since a bigger model could just set the new coefficient to zero and keep the old fit.

(b) Theoretical averages

For B_m^2, I fit the true mean μ with each smaller model. Then I used the part that was still left over. I did it this way so I did not have to make the full hat matrix.

```
m_grid <- 0:p
bias2 <- numeric(p + 1)

for (m in m_grid) {
  mu_fit <- lm.fit(make_X(m), mu)$fitted.values
  bias2[m + 1] <- sum((mu - mu_fit)^2)
}

theory_train <- bias2 / n + 1 - (m_grid + 1) / n
theory_test <- bias2 / n + 1 + (m_grid + 1) / n

all_values <- c(
  avg_train, avg_test, theory_train,
  theory_test, one_test_mse
)

plot(
  m_grid, one_test_mse,
  type = "l", lty = 3, col = "grey55",
  xlab = "Number of predictors (m)",
  ylab = "MSE",
  ylim = range(all_values)
)
lines(m_grid, avg_train, col = "blue", lwd = 2)
lines(m_grid, avg_test, col = "red", lwd = 2)
lines(m_grid, theory_train, col = "blue", lty = 2, lwd = 2)
lines(m_grid, theory_test, col = "red", lty = 2, lwd = 2)
legend(
  "topright",
  c("one test run", "average train", "average test",
    "theory train", "theory test"),
  col = c("grey55", "blue", "red", "blue", "red"),
  lty = c(3, 1, 1, 2, 2),
  lwd = c(1, 2, 2, 2, 2),
```

```
  cex = 0.72,
  bty = "n"
)
```



The solid lines and dashed lines are pretty close and have the same basic shape.

(c) What the curves mean

The dotted line is only one test run, and it looks a lot more jumpy. That one line depends on the random noise it happened to get. The average of 200 runs gets rid of a lot of those random jumps.

There are basically two things in test error. Leaving out useful predictors gives bias, but adding more coefficients adds variance. Here both test curves hit their lowest point at $m = 6$. Training MSE keeps falling after that, so training MSE by itself would pick too many predictors.

Question 2

(a) Theoretical error at the target point

```
n2 <- 100
p2 <- 6
sigma2 <- 1

i <- 1:n2
X2 <- outer(
  i, 1:p2,
  function(i, j) sqrt(2) * cos(pi * j * (i - 0.5) / n2)
)

beta2 <- rep(0.5, p2)
x0 <- c(0, 1, 0, 0, 1, 0)
mu2 <- as.vector(X2 %*% beta2)
mu0 <- sum(x0 * beta2)
m2 <- 0:p2

sq_bias <- sapply(m2, function(m) {
```

```r
  omitted <- if (m == p2) numeric(0) else (m + 1):p2
  sum(x0[omitted] * beta2[omitted])^2
})

variance <- sapply(m2, function(m) {
  included <- if (m == 0) numeric(0) else 1:m
  sigma2 / n2 * (1 + sum(x0[included]^2))
})

theory2 <- sq_bias + variance
theory_table <- data.frame(
  m = m2,
  squared_bias = sq_bias,
  variance = variance,
  expected_error = theory2
)

cat("mu_0 =", mu0, "\n")
```

```
## mu_0 = 1
```

```r
knitr::kable(theory_table, digits = 2)
```

m	squared_bias	variance	expected_error
0	1.00	0.01	1.01
1	1.00	0.01	1.01
2	0.25	0.02	0.27
3	0.25	0.02	0.27
4	0.25	0.02	0.27
5	0.00	0.03	0.03
6	0.00	0.03	0.03

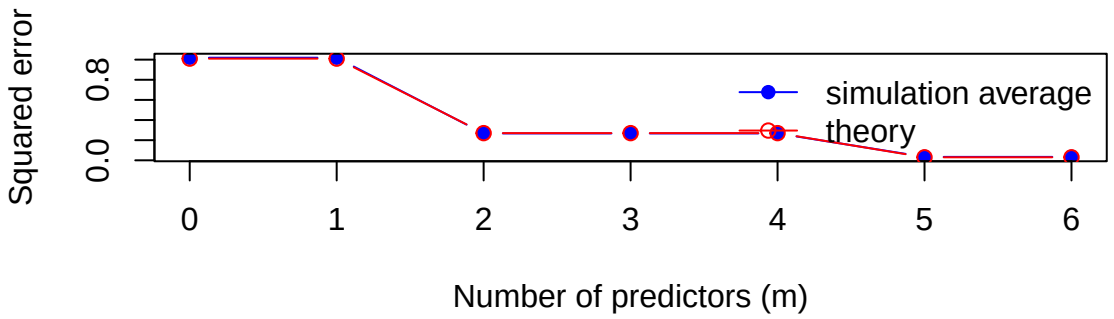
Here μ_0 comes out to 1. The bias part is what is missing when a needed predictor is not in the model. The variance part comes from estimating the intercept and coefficients with noisy data.

The error is the same at $m = 0$ and 1. It changes at $m = 2$, stays the same up to $m = 4$, and changes again at $m = 5$. It does not change at $m = 6$. The changes happen when predictor 2 or predictor 5 gets added.

(b) Simulation

```r
set.seed(43203)

sq_error <- matrix(NA_real_, 200, p2 + 1)

for (b in 1:200) {
  y <- mu2 + rnorm(n2)

  for (m in 0:p2) {
    Xm <- cbind(1, X2[, seq_len(m), drop = FALSE])
    fit <- lm.fit(Xm, y)
    target_row <- c(1, x0[seq_len(m)])
    mu0_hat <- sum(target_row * fit$coefficients)
    sq_error[b, m + 1] <- (mu0_hat - mu0)^2
  }
}
```

```r
sim_average <- colMeans(sq_error)

plot(
  m2, sim_average,
  type = "b", pch = 16, col = "blue",
  xlab = "Number of predictors (m)",
  ylab = "Squared error",
  ylim = range(c(sim_average, theory2))
)
lines(m2, theory2, type = "b", pch = 1, col = "red")
legend(
  "topright",
  c("simulation average", "theory"),
  col = c("blue", "red"),
  lty = 1,
  pch = c(16, 1),
  bty = "n"
)
```




```r
cat("Smallest theoretical minimizer: m =",
    which.min(theory2) - 1, "\n")
```

```
## Smallest theoretical minimizer: m = 5
```

The blue simulation points are pretty close to the red theory points. The first m with the smallest theoretical error is $m = 5$.

(c) Why only two predictors matter here

Every beta is 0.5, but $\mathbf{x}_0$ is only nonzero at positions 2 and 5. So in $\mathbf{x}_0^{\mathsf{T}}\beta$, all the other terms are just zero.

Question 1 is different because its test MSE uses all rows of the design. A predictor could matter for some of those rows even when it does nothing at this one point.

Question 3

(a) Training and test expectations

I first write the training residual as

$$\mathbf{y} - \mathbf{H}\mathbf{y} = (\mathbf{I} - \mathbf{H})\mu + (\mathbf{I} - \mathbf{H})\epsilon.$$

The cross term goes away after taking expectation because the error has mean zero. Also, $(\mathbf{I}-\mathbf{H})^2 = \mathbf{I}-\mathbf{H}$. So

$$\begin{aligned} E\left(\|\mathbf{y}-\mathbf{H}\mathbf{y}\|^2 \mid \mathbf{X}\right) &= B^2 + \sigma^2 \operatorname{tr}(\mathbf{I}-\mathbf{H}) \\ &= B^2 + \sigma^2\{n-(p+1)\}. \end{aligned}$$

Then I divide by n:

$$E(\mathrm{MSE}_{\mathrm{train}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 - \frac{p+1}{n}\right).$$

For the test response, the residual is

$$\mathbf{y}^* - \mathbf{H}\mathbf{y} = (\mathbf{I}-\mathbf{H})\mu + \epsilon^* - \mathbf{H}\epsilon.$$

The two error vectors are independent, so the cross terms are zero after taking expectation. One variance part is $\sigma^2 n$, and the other is $\sigma^2(p+1)$. This gives

$$E(\mathrm{MSE}_{\mathrm{test}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 + \frac{p+1}{n}\right).$$

(b) Expected optimism

I subtract the training result from the test result:

$$E(\mathrm{MSE}_{\mathrm{test}} - \mathrm{MSE}_{\mathrm{train}} \mid \mathbf{X}) = \frac{2\sigma^2(p+1)}{n}.$$

For the numbers in the question,

$$\begin{aligned} E(\mathrm{MSE}_{\mathrm{train}} \mid \mathbf{X}) &= 0.04 + 1 - \frac{5}{120} = 0.9983, \\ E(\mathrm{MSE}_{\mathrm{test}} \mid \mathbf{X}) &= 0.04 + 1 + \frac{5}{120} = 1.0817, \\ \text{difference} &= \frac{10}{120} = 0.0833. \end{aligned}$$

These numbers are averages over possible random errors. For one actual run, the noise can still make test MSE smaller than training MSE.

(c) Corrected MSE and C_p

$$\widehat{\mathrm{MSE}}_{\mathrm{test}} = \frac{116.4 + 2(5)(0.96)}{120} = 1.05,$$

and

$$C_p = \frac{116.4}{0.96} - 120 + 2(5) = 11.25.$$

Moving the terms around in the C_p formula gives

$$\widehat{\mathrm{MSE}}_{\mathrm{test}} = \frac{\hat{\sigma}^2}{n}(C_p + n).$$

The same n and $\hat{\sigma}^2$ are used for every model. So smaller C_p also means smaller corrected test MSE here.

Question 4

(a) Fits and common variance estimate

```
diabetes <- read.csv("data/diabetes.csv")
train <- diabetes[1:370, ]

fit_A <- lm(
  y ~ bmi + bp + s5 + sex + s1 + s2 + s4,
  data = train
)
fit_B <- lm(
  y ~ bmi + bp + s5 + sex + s1 + s2 + s4 + s6,
  data = train
)
fit_full <- lm(y ~ ., data = train)

rss <- c(
  "Model A" = deviance(fit_A),
  "Model B" = deviance(fit_B),
  "Full reference" = deviance(fit_full)
)
p_count <- c(7, 8, 10)
full_df <- df.residual(fit_full)
sigma_hat2 <- rss["Full reference"] / full_df

fit_table <- data.frame(
  model = names(rss),
  p = p_count,
  RSS = unname(rss)
)

knitr::kable(fit_table, digits = 2)
```

model	p	RSS
Model A	7	1098909
Model B	8	1089717
Full reference	10	1089452

```
cat("Full model residual degrees of freedom:", full_df, "\n")
```

```
## Full model residual degrees of freedom: 359
```

```
cat("Common variance estimate:",
    round(sigma_hat2, 3), "\n")
```

```
## Common variance estimate: 3034.684
```

I only used the first 370 rows for this part. I got 359 residual degrees of freedom for the full model. The common variance estimate is 3034.684.

(b) Three criteria

```r
n_train <- 370
p_candidates <- c("Model A" = 7, "Model B" = 8)
candidate_rss <- rss[c("Model A", "Model B")]
k <- p_candidates + 1

cp <- candidate_rss / sigma_hat2 - n_train + 2 * k
aic_red <- n_train * log(candidate_rss / n_train) + 2 * k
bic_red <- n_train * log(candidate_rss / n_train) +
  k * log(n_train)

criteria_table <- data.frame(
  model = names(candidate_rss),
  Cp = unname(cp),
  reduced_AIC = unname(aic_red),
  reduced_BIC = unname(bic_red)
)

knitr::kable(criteria_table, digits = 3)
```

model	Cp	reduced_AIC	reduced_BIC
Model A	8.116	2974.640	3005.948
Model B	7.087	2973.532	3008.754

For C_p, Model B has the smaller number, 7.087 instead of 8.116. Reduced AIC also picks Model B. Reduced BIC goes the other way and picks Model A. I only compared the two models inside each column.

(c) Why the criteria disagree

Adding s6 lowers RSS by 9191.509. On the C_p scale, that is an improvement of $9191.509/3034.684 = 3.029$. This is more than the penalty of 2. The AIC improvement is 3.108, which is also more than 2. So these two pick Model B.

BIC has a bigger penalty, $\log(370) = 5.914$. The improvement is not enough for that penalty, so BIC stays with Model A.

This still does not mean Model A or B has to be the true model. It only means one of them got the better score out of the choices given here.

Question 5

(a) Misuse of the final test data

The first problem is step 2. All eleven models are checked on the final test data so one can be chosen later. This already makes the test set work like validation data. It does not matter that the coefficients were fit somewhere else.

Each test MSE has some luck in it. If I look at eleven of them and keep the smallest one, I will probably also keep some lucky noise. That makes the reported minimum look better than future data probably will.

(b) Five-fold cross-validation

I would split the 370 training rows into five folds. For each $m = 0, \ldots, 10$, I would fit on four folds and get MSE on the fold left out. Then I would average the five MSEs and pick the smallest one. I would refit that model on all 370 rows, and only then use rows 371 through 442 one time for the final test MSE.

(c) If the test results were already viewed

All eleven numbers can still be shown, but it must be clear that the test set was used to compare models. So the minimum is not a new final check. For a new one, another independent data set is needed, and it should stay unseen until the whole model procedure is decided.